实验六 RYU 控制器编程实践

一、 教学目的与学时建议

1. 能够在 Linux 系统下安装、配置 RYU 控制器编程与运行环境；

2. 能够对 SDN 流量分析、路由器和防火墙等网络功能需求及关键问题进行梳理，并进行方案设计；

3. 能够基于 RYU 控制器开源代码库，开发流量分析、路由器和防火墙等网络应用组件；

4. 能够调试和运行所编写的网络应用组件程序，并对结果进行分析与评判

建议：4 学时

二、 技术与知识背景

1. RYU

Ryu 是一个基于组件的软件定义网络框架。Ryu 为软件组件提供定义良好的 API，使开发人员可以轻松创建新的网络管理和控制应用程序。Ryu 支持各种用 于管理网络设备的协议，例如 OpenFlow，Netconf，OF-config 等。关于 OpenFlow， Ryu 支持完全支持 1.0,1.2,1.3,1.4,1.5 和 Nicira Extensions。

2. RYU 文件目录

RYU 目录结构如图 1 所示

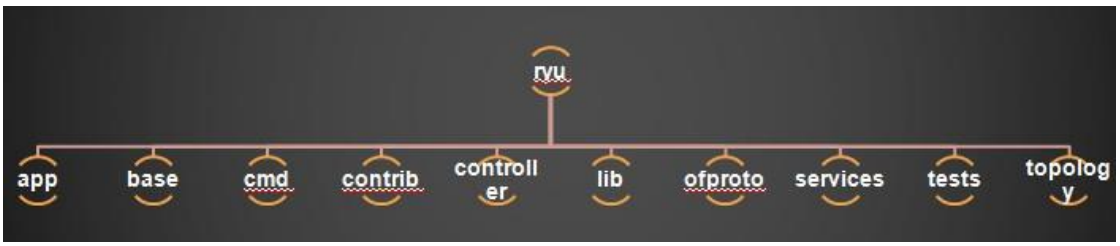


图 1

- **base**

base 中有一个非常重要的文件：app_manager.py，其作用是 RYU 应用的管理中心。用于加载 RYU 应用程序，接受从 APP 发送过来的信息，同时也完成消 息的路由。

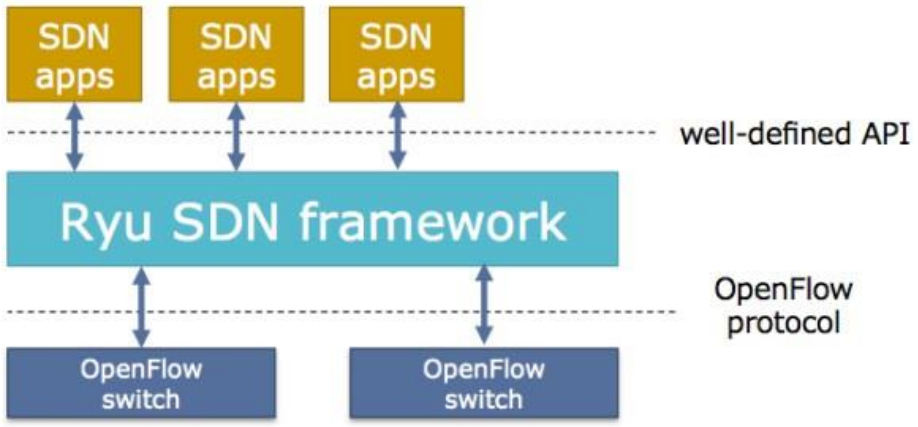
其主要的函数有 app 注册、注销、查找、并定义了 RYUAPP 基类，定义了 RYUAPP 的基本属性。包含 name, threads, events, event_handlers 和 observers 等 成员，以及对应的许多基本函数。如：start(), stop()等。

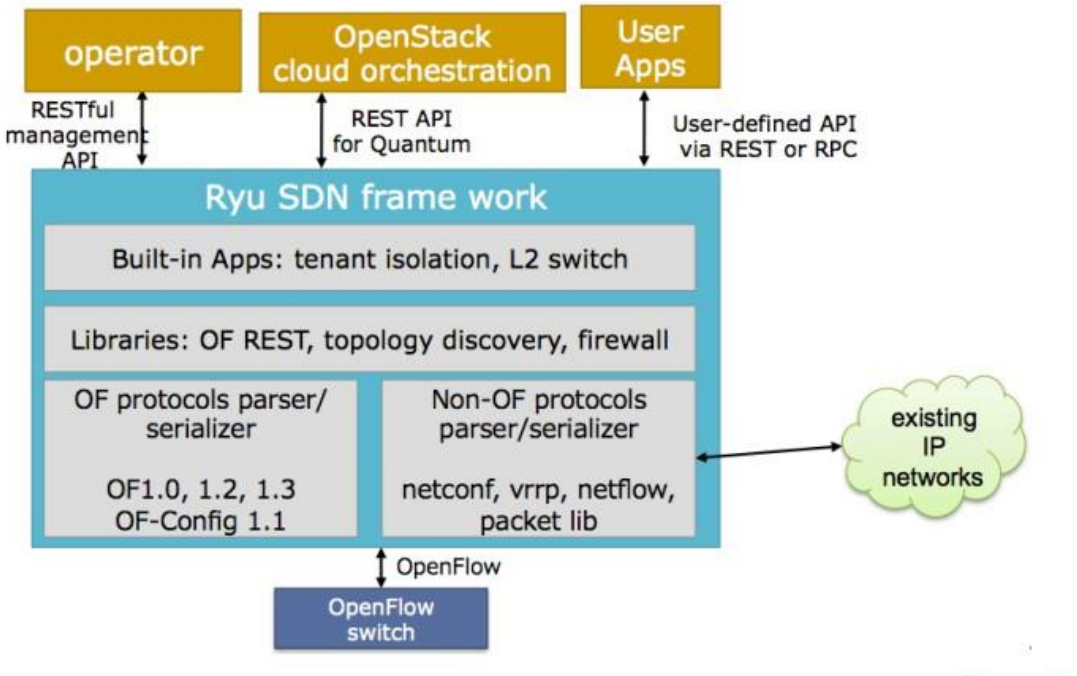
- **controller**

controller 文件夹中许多非常重要的文件，如 events.py, ofp_handler.py, controller.py 等。其中 controller.py 中定义了 OpenFlowController 基类。用于定义

OpenFlow 的控制器，用于处理交换机和控制器的连接等事件，同时还可以产生事件和路由事件。其事件系统的定义，可以查看 events.py 和 ofp_events.py。

3. RYU 架构

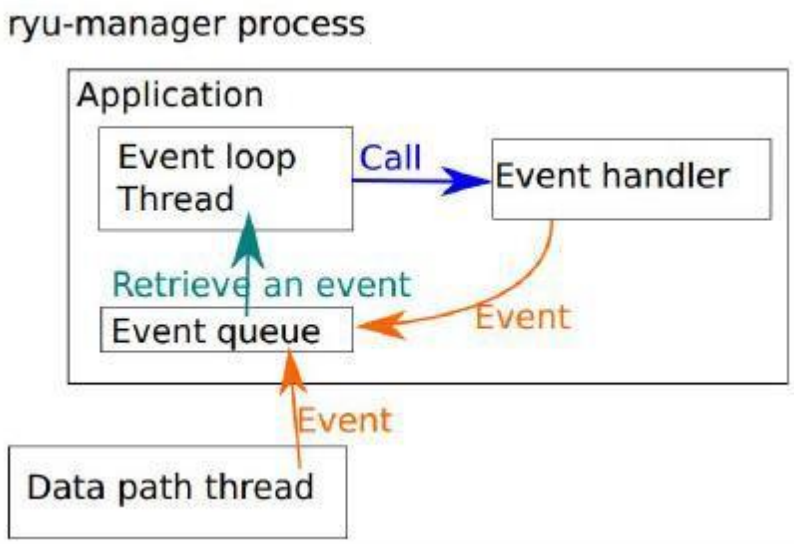




各个组件的功能：

组件类型及说明	组件名	功能
基本组件	ryu/base/app_manager.py	提供了对其他组件的管理，被ryu-manager自动调用
	ryu/controller/dpset.py	管理OF交换机的组件，今后的版本中可能会被ryu/topology代替
	ryu/controller/ofp_handler.py	对控制器、交换机间握手、协商过程的处理
	ryu/controller/ofp_event.py	完成OF消息-事件的转化，提供北向接口API
	ryu/controller/controller.py	控制器组件，管理与OF交换机连接的安全通道，接收OF消息，调用ofp_event，并发布相应的“事件”，以触发订阅了该“事件”的组件的处理逻辑
与Of协议相关的组件	Ofproto1x.py(x=0/1/2/3)	定义了相应协议版本的参数
	Ofproto1x_parser.py(x=0/1/2/3)	定义了相关协议版本消息的封装格式
应用类组件	ryu/app/simple_switch.py	传统的2层交换机策略
	ryu/app/simple_switch_stp.py	无广播环路的2层交换机策略
	ryu/app/simple_isolation.py	基于MAC的过滤策略
	ryu/app/simple_vlan.py	实现了基本的VLAN
	ryu/app/gre_tunnel.py	实现了多种隧道策略
	ryu/topology.py	交换机与链路状态监测组件，用于拓扑图的构建
基于REST API的应用组件	ryu/app/ofctl_rest.py	用于管理OVS中的流表
	ryu/app/rest_conf_switch.py	用于配置OF交换机
	ryu/app/rest_quantum.py	与Quantum的通信接口
	ryu/app/rest_router.py	路由策略
	ryu/app/rest_topology.py	用于对拓扑的管理
	ryu/app/rest_tunnel.py	隧道策略

4. RYU 应用程序编程模式



Ryu 事件处理、进程与线程：

（1) Applications：该类继承自 ryu.base.app_manager.RyuApp，用户逻辑被描 述为一个 APP。

（2) Event ： 继承自 ryu.controller.event.EventBase ， 应用程序之间的通信 由 transmitting and receiving events 完成。

（3) Event Queue：每一个 application 都有一个队列用于接收事件。

（4) Threads：Ryu 使用第三方库 eventlets 运行多线程。因为线程是非抢占 式的，因此，当执行耗时的处理程序时要非常小心。

（5) Event loops： 创建一个 application 时，会自动生成一个线程，该线程运行一个事件循环。当队列事件不为空时，这个事件循环会加载该事件并且调用 相应的事件处理函数（注册之后）。

（6) Additional threads：可以使用 hub.spawn()添加其它线程，用来处理特殊 的应用

（7) Eventlets：这是一个第三方库，里面的库函数被封装到 hub 模块中被开 发人员加载使用。

（8) Event handlers：使用 ryu.controller.handler.set_ev_cls 修饰一个事件处理 函数。当该类型的事件触发后，事件处理函数就会被应用程序的事件循环调用。

三、 基础技能

1. 内容与任务

1) 从 ftp 中拷贝 example_switch.py 文件，在了解了已给出代码的含义条件下， 结合相关注释完成 example_switch.py 脚本的完善，完成下列注释下的代码 编写，实现交换机功能。

#decide which port to send the packets, otherwise, flood.

#contruct actions

#install a flow mod msg

send a packet out.

2) 使用 mininet 对交换机功能进行验证。在 mininet 中运行单交换机，3 主机的星形拓扑，测试联通性，截图至实验报告；然后从 h1 发包至 h2，在 h3 终端中运行 wireshark，观察是否能抓到该数据包，截图至实验报告，并说明原因。（注意：可通过 sudo mn –controller=remote – mac 对 mac 地址按序编号打印，方便解读打印内容。）

```
EVENT ofp_event->ExampleSwitch EventOFPSwitchFeatures
switch features ev version=0x4,msg_type=0x6,msg_len=0x20,xid=0xcb502a90,OFPSwit
chFeatures(auxiliary_id=0,capabilities=79,datapath_id=1,n_buffers=0,n_tables=25
4)
move onto main mode
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:02 33:33:ff:00:00:02 2
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:01 33:33:ff:00:00:01 1
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
packet in 1 00:00:00:00:00:02 33:33:00:00:00:02 2
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
packet in 1 00:00:00:00:00:01 33:33:00:00:00:02 1
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:02 33:33:00:00:00:16 2
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:01 33:33:00:00:00:16 1
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
packet in 1 00:00:00:00:00:01 ff:ff:ff:ff:ff:ff 1
EVENT ofp_event->ExampleSwitch EventOFPPacketIn
```

```
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
```

3）参考交换机的脚本，实现普通集线器的功能 hub.py 脚本，它从每一个端口收到的数据包，向所有其他端口洪泛转发该数据包。在 mininet 中运行单交换机，3 主机的星形拓扑，从 h1 发包至 h2，在 h3 终端中运行 wireshark，观察是否能抓到该数据包，截图至实验报告，并说明原因。

提示与建议

需要根据 dst 是否存在与 mac_to_port[dpid]子字典中，对数据包的动作区分

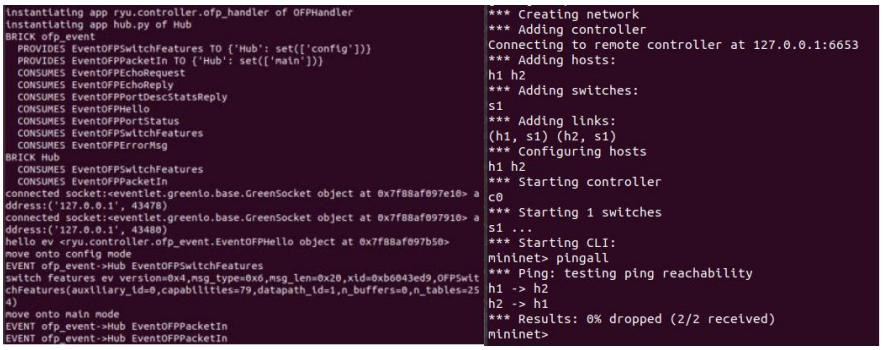
为指定端口转发或者是泛洪转发。即：

```
#decide which port to send the packets, otherwise, flood. if dst in
self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
else:
        out_port = ofproto.OFPP_FLOOD
#contruct actions.
actions = [ofp_parser.OFPActionOutput(out_port)]
#install a flow mod msg.
```

当动作 actions 为泛洪时候，则不需要下发任何流表，只有当 out_port 为一 个具体的值才需要下发一个具体的流表。因此需要对 out_port 进行一次判断。

```
# send a packet out.
```

相比于 hub.py 中代码需要在构建‘out’时，函数 OFPPacketOut()中需要添 加新的参数 data，data 属性值为 msg.data。



四、 进阶技能

1）在上述交换机代码基础上，实现防火墙组件 firewall.py，该组件可根据给定 的规则 firewall-policies.csv 文件，对源和目的 MAC 地址进行匹配，对匹配成功 的数据进行丢弃。

2）用 mininet 搭建如下拓扑，然后运行 ryu 目录下的 rest_firewall，用 restAPI 请求，实现 h1 和 h2 能相互 ping 通，导出截图 s1 的流表。然后再通过加入高优先级防火墙规则实现 h1 和 h2 的 ping 阻断。（参考 RYUBOOK 中的防火墙例子实现）

